



Travaux Dirigés

Jeu de Memory

Conception, Déploiement et Documentation d'un Jeu de Memory en Vanilla JS

Cadre architectural & Objectif final

L'objectif de ce TD est de concevoir une application web interactive complète **en séparant strictement la logique algorithmique** (modèle de données) **de l'interface graphique** (DOM).

Dès le départ, vous devez savoir que ce projet a pour vocation d'être **déployé publiquement en production via GitHub Pages** et **documenté** de manière professionnelle dans un **fichier README.md**.

À chaque fin d'étape, vous devez impérativement **valider vos modifications localement** ET les **pousser sur votre serveur distant**.

Étape 1 : Initialisation des données et algorithmique pure

1. **Workspace** : Configurez vos dossiers sous VS Code (`index.html`, `CSS/style.css`, `JS/script.js`, `favicon.png`). Activez Live Server et ouvrez la Console (F12) comme outil de débogage.
2. **Génération d'images** : Déclarez deux variables globales : `dimension` (150) et `imgStart` (entier aléatoire de 1 à 100). Via une boucle `for`, stockez 8 URL uniques de l'API **Picsum** (<https://picsum.photos/>) en exploitant obligatoirement les **Littéraux de gabarit** (``` et `$`{}) et les deux variables créées.
3. **Duplication** : Créez le tableau `cards` en dupliquant `images` avec le Spread Operator : `let cards = [...images, ...images];`
4. **Mélange** : Écrivez la fonction `shuffle(array)`. Implémentez l'algorithme de permutation de **Fisher-Yates** en utilisant la boucle inversée et la permutation par déstructuration en une ligne.

Synchronisation Git obligatoire :

```
git add .
git commit -m "feat: core array data logic and fisher-yates shuffle functional"
git push origin main
```

Étape 2 : Manipulation du DOM et rendu dynamique

5. **Encapsulation** : Déclarez `initGame()` et déplacez-y l'appel à `shuffle(cards);`.



6. **Rendu dynamique** : Utilisez `.forEach()` sur `cards` pour générer pour chaque URL un élément HTML `<div>` doté de la classe CSS `'card'`.
7. **Dataset** : Encapsulez l'URL secrète dans la propriété `card.dataset.value = imgUrl;`
8. **Inclusion & A11y** : Injectez les attributs d'accessibilité indispensables `role='button'` et `tabindex='0'` sur chaque carte avant de l'insérer dans le nœud `#game-board` via `.appendChild()`.

Synchronisation Git obligatoire :

```
git add .
git commit -m "feat: dynamic DOM generation for game board with data-attributes
and a11y properties"
git push origin main
```

Étape 3 : Gestion des événements et logique applicative (Asynchronisme)

9. **Variables de scope** : Initialisez au sommet du script les variables de suivi : `firstCard (null)`, `secondCard (null)`, `lockBoard (false)`, `moves (0)` et `matchedCount (0)`.
10. **Écoute** : Dans votre boucle de génération des cartes, ajoutez l'écouteur d'événement : `card.addEventListener('click', () => handleCardClick(card));`
11. **Vérifications logiques** : Écrivez `handleCardClick(card)`. Empêchez toute action illégale (double clic, carte déjà découverte, plateau verrouillé) via une structure de garde (`return;`).
12. **Gestion des tours** : Au premier clic, stockez la carte dans `firstCard`. Au second clic, assignez `secondCard`, verrouillez le plateau (`lockBoard = true`), incrémentez `moves` et invoquez `checkMatch()`.
13. **Asynchronisme** : Dans `checkMatch()`, si les valeurs concordent, ajoutez la classe `'matched'`. Si elles diffèrent, utilisez un `setTimeout` de 800 ms pour vider le contenu interne des cartes (`.innerHTML = ''`) afin de masquer les images avant de réinitialiser le tour.

Synchronisation Git obligatoire :

```
git add .
git commit -m "feat: complete card click handling, dynamic image rendering, and
asynchronous match validation"
git push origin main
```

Étape 4 : Cycle de fin de partie, chronomètre et réinitialisation synchrone

14. **Variables d'état** : Déclarez globalement `let seconds = 0;` et `let timerInterval = null;`
15. **Formatage** : Écrivez `formatTime(sec)` qui renvoie une chaîne `mm:ss` en utilisant `String().padStart(2, '0')` pour gérer les zéros initiaux.



16. **Chronomètre** : Écrivez `startTimer()` incrémentant `seconds` toutes les 1000 ms via un `setInterval` et modifiant `timerDisplay.textContent`.
17. **Condition de victoire** : Écrivez `checkVictory()`. Si `matchedCount === cards.length`, stoppez le temps (`clearInterval`) et affichez le score dans `#result`.
18. **Reset Synchrone** : Modifiez `initGame()` pour vider le plateau (`board.innerHTML = ''`), réinitialiser tous les compteurs globaux à zéro, effacer l'intervalle actif et relancer `startTimer()`.

Étape 5 : Déploiement GitHub Pages et Valorisation du Portfolio

19. **Hébergement public** : Connectez-vous à GitHub. Créez un dépôt public nommé 'js-memory-game'. Liez votre dépôt local et effectuez votre premier push global.
20. **Configuration GitHub Pages** : Dans l'onglet Settings -> Pages de votre dépôt distant, sélectionnez une option 'Deploy from a branch'. Choisissez la branche principale (main ou master) et validez. Votre jeu est désormais disponible mondialement à une adresse publique !
21. **Rédaction du README.md** : À la racine de votre projet, concevez un fichier `README.md` rédigé en Markdown incluant : le titre avec le lien de production, une description claire des technos (Vanilla JS ES6, CSS Grid), la liste des fonctionnalités (A11y ARIA, Fisher-Yates, asynchronisme) et un guide de lancement local.

Synchronisation Git obligatoire :

```
git add .
git commit -m "feat: implement timer lifecycle, complete application deployment
and readme documentation"
git push origin main
```

Aide et ressources utiles :

Vous pouvez utiliser l'**interface graphique de VSC** pour le versionnage du code si vous ne souhaitez pas le faire en lignes de commande.

Le support de cours Javascript présente la grande majorité des notions nécessaires à la réalisation du TP, n'hésitez pas à le consulter.

Aria-label :

<https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Reference/Attributes/aria-label>

Attributs de données et Dataset :

https://developer.mozilla.org/fr/docs/Web/HTML/How_to/Use_data_attributes

A11y (Accessibilité) :

<https://developer.mozilla.org/fr/docs/Web/Accessibility>

Algorithme de permutation de Fisher-Yates :

<https://labex.io/fr/tutorials/shuffling-arrays-with-fisher-yates-algorithm-28615>